Smart Parking IoT System

Systeme de Gestion de Parking Intelligent

Base sur ESP32, MQTT, MongoDB, Redis et RAG-LLM

ESP32 | **MQTT** | **HiveMQ** | **MongoDB** | **Redis** | **Docker** | **RAG-LLM**

Realise par : DRISSI Yassir
BOUSHABI Othmane
DHILOU Othmane
DBAB Mohamed

Encadrant : DEROUSSI Anass

Annee academique : 2025 – 2026

Table des matières

1 Introduction Generale **4**

1.1 Contexte du Projet 4

1.2 Objectifs du Projet 4

1.3 Organisation du Rapport 5

2 Architecture et Technologies Utilisees **6**

2.1 Architecture Globale 6

2.2 Technologies Utilisees 6

3 Conception Materielle **8**

3.1 Presentation des Composants 8

3.1.1 Microcontroleur ESP32 8

3.1.2 Capteur Ultrasonique HC-SR04 8

3.1.3 Servo Moteur SG90 8

3.2 Brochage et Cablage 9

3.3 Simulation sur Wokwi 9

4 Developpement du Firmware ESP32 **11**

4.1 Structure du Code 11

4.2 Librairies et Configuration 11

4.3 Fonction de Mesure de Distance 12

4.4 Logique Principale 12

4.5 Test de la Barriere 13

5 Communication MQTT **14**

5.1 Presentation du Protocole MQTT 14

5.2 Configuration HiveMQ 14

5.3 Format des Messages 14

5.4 Visualisation des Messages 15

6 Backend – Docker, MongoDB et Redis **16**

6.1 Architecture Docker 16

6.2 Services Docker 16
6.3 Resultats 17

7 Dashboard Web de Visualisation 19
7.1 Presentation 19
7.2 Vue Generale 19
7.3 Graphiques 20
7.4 Fonctionnalites 20

8 Integration RAG-LLM 21
8.1 Presentation du RAG 21
8.2 Architecture RAG 21
8.3 Implementation 21
8.4 Technologies IA 22

9 Conclusion et Perspectives 23
9.1 Bilan du Projet 23
9.2 Competences Acquises 23
9.3 Perspectives et Ameliorations 24
9.4 Conclusion 24

Table des figures

3.1 Montage complet sur Wokwi – ESP32, 3 capteurs HC-SR04 et servo moteur . . . 9
3.2 Simulation en cours – Serial Monitor avec messages MQTT publies . 10

4.1 Test : place libre + voiture a la barriere – Servo 90 degres (OUVERT) 13

5.1 Interface HiveMQ WebSocket Client . . . 15
5.2 Messages MQTT recus en temps reel sur le topic `parking/update` . . 15

6.1 Fichier docker-compose.yml – Orchestration des services . . . 16
6.2 Docker Desktop – Les 3 containers en cours d'execution . . . 17
6.3 Logs Docker – Messages MQTT recus, sauvegardes dans MongoDB et Redis . . . 17
6.4 Donnees sauvegardees dans MongoDB . . . 18

7.1 Dashboard Smart Parking IoT – Vue principale . . . 19
7.2 Dashboard – Historique d'occupation et repartition actuelle . . . 20

Chapitre 1

Introduction Generale

1.1 Contexte du Projet

L'essor des objets connectes (IoT) transforme profondement notre quotidien. Parmi les defis urbains modernes, la gestion du stationnement represente une problematique majeure dans les villes et les etablissements. Ce projet propose un **systeme de parking intelligent** capable de detecter en temps reel l'etat des places de stationnement, de controler automatiquement l'acces, et de fournir une visualisation complete via un tableau de bord web.

1.2 Objectifs du Projet

Ce projet vise a :

- Detecter automatiquement l'occupation des places via des capteurs ultrasoniques
- Controler une barriere d'entree automatique via un servo moteur
- Transmettre les donnees en temps reel via le protocole MQTT
- Stocker l'historique dans une base de donnees MongoDB
- Mettre en cache les donnees frequentes avec Redis
- Visualiser l'etat du parking via un dashboard web interactif
- Integrer un assistant IA (RAG-LLM) pour des recommandations intelligentes

1.3 Organisation du Rapport

1. **Chapitre 1** : Introduction Generale
2. **Chapitre 2** : Architecture et Technologies Utilisees
3. **Chapitre 3** : Conception Materielle
4. **Chapitre 4** : Developpement du Firmware ESP32
5. **Chapitre 5** : Communication MQTT
6. **Chapitre 6** : Backend – Docker, MongoDB et Redis
7. **Chapitre 7** : Dashboard Web de Visualisation
8. **Chapitre 8** : Integration RAG-LLM
9. **Chapitre 9** : Conclusion et Perspectives

Chapitre 2

Architecture et Technologies Utilisees

2.1 Architecture Globale

Le systeme repose sur une architecture IoT en trois couches :

```
+-----------------------------------------------------+
|                   COUCHE TERRAIN                    |
|  HC-SR04 #1   HC-SR04 #2   HC-SR04 #3   Servo SG90 |
|  (Place 1)    (Place 2)    (Barriere)               |
+-----------------------------------------------------+
                       |  GPIO
+-----------------------------------------------------+
|              COUCHE OBJET CONNECTE                  |
|                    ESP32                            |
|          WiFi + MQTT + PWM + Arduino C++            |
+-----------------------------------------------------+
                       |  WiFi / TCP-IP
+-----------------------------------------------------+
|                COUCHE CLOUD / SERVICES              |
| HiveMQ Broker -> Backend Node.js -> MongoDB + Redis |
|                    Dashboard HTML                   |
|                   RAG-LLM (IA)                      |
+-----------------------------------------------------+
```

2.2 Technologies Utilisees

Technologie	Categorie	Role dans le projet
ESP32 DevKit	Microcontroleur	Cerveau du systeme, WiFi integre
HC-SR04	Capteur	Detection de presence par ultra-sons
Servo SG90	Actionneur	Controle de la barriere d'entree
Arduino C++	Langage	Programmation du firmware ESP32
Wokwi	Simulateur	Simulation IoT en ligne
MQTT	Protocole	Communication IoT legere
HiveMQ	Broker MQTT	Serveur de messagerie public
Node.js	Backend	Serveur de traitement des donnees
MongoDB	Base de donnees	Stockage historique des evenements
Redis	Cache	Acces rapide a l'etat actuel
Docker	Conteneurisation	Orchestration des services
HTML/CSS/JS	Frontend	Dashboard de visualisation
Chart.js	Visualisation	Graphiques temps reel
OpenRouter	IA	API LLM pour RAG
Llama 3.2	Modele LLM	Recommandations intelligentes

TABLE 2.1 – Technologies utilisees dans le projet

Chapitre 3

Conception Materielle

3.1 Presentation des Composants

3.1.1 Microcontroleur ESP32

L'ESP32 est un microcontroleur haute performance fabrique par Espressif Systems. Ses caracteristiques principales :

— Processeur dual-core a 240 MHz

— WiFi 802.11 b/g/n integre

— 520 KB de RAM

— 34 broches GPIO programmables

— Support PWM, I2C, SPI, UART

3.1.2 Capteur Ultrasonique HC-SR04

Le capteur HC-SR04 mesure la distance par reflexion d'ondes ultrasoniques :

$$d = \frac{t_{echo} \times v_{son}}{2} = \frac{t_{echo} \times 0{,}034}{2} \quad \text{[cm]}$$

3.1.3 Servo Moteur SG90

Le servo SG90 controle la barriere d'entree via un signal PWM a 50 Hz :

— $0°$: barriere **fermee**

— $90°$: barriere **ouverte**

3.2 Brochage et Cablage

Composant	Broche TRIG	Broche ECHO	Role
HC-SR04 #1	GPIO 5	GPIO 18	Place de parking 1
HC-SR04 #2	GPIO 19	GPIO 21	Place de parking 2
HC-SR04 #3	GPIO 23	GPIO 22	Capteur barriere
Servo SG90	GPIO 27 (PWM)	—	Barriere d'entree

TABLE 3.1 – Tableau de brochage ESP32

3.3 Simulation sur Wokwi

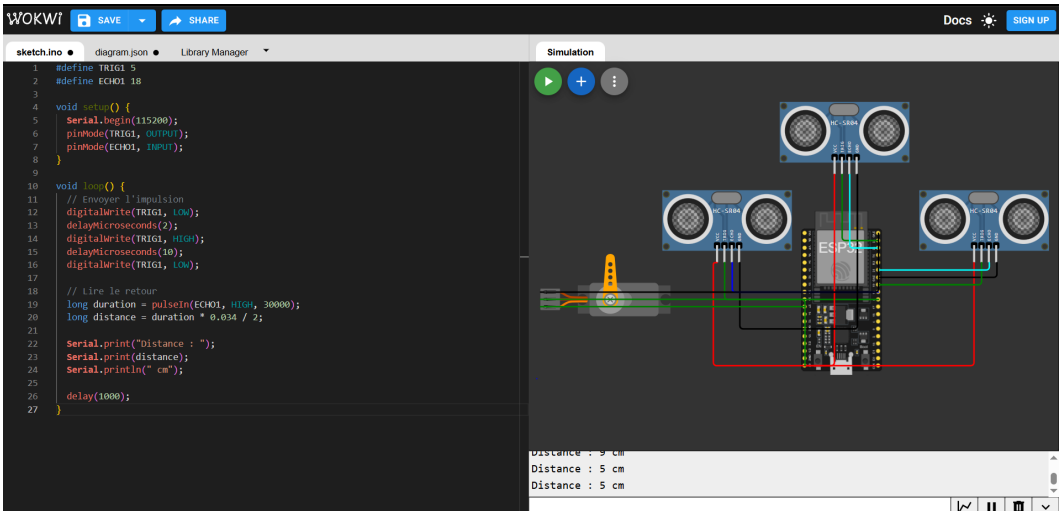

FIGURE 3.1 – Montage complet sur Wokwi – ESP32, 3 capteurs HC-SR04 et servo moteur

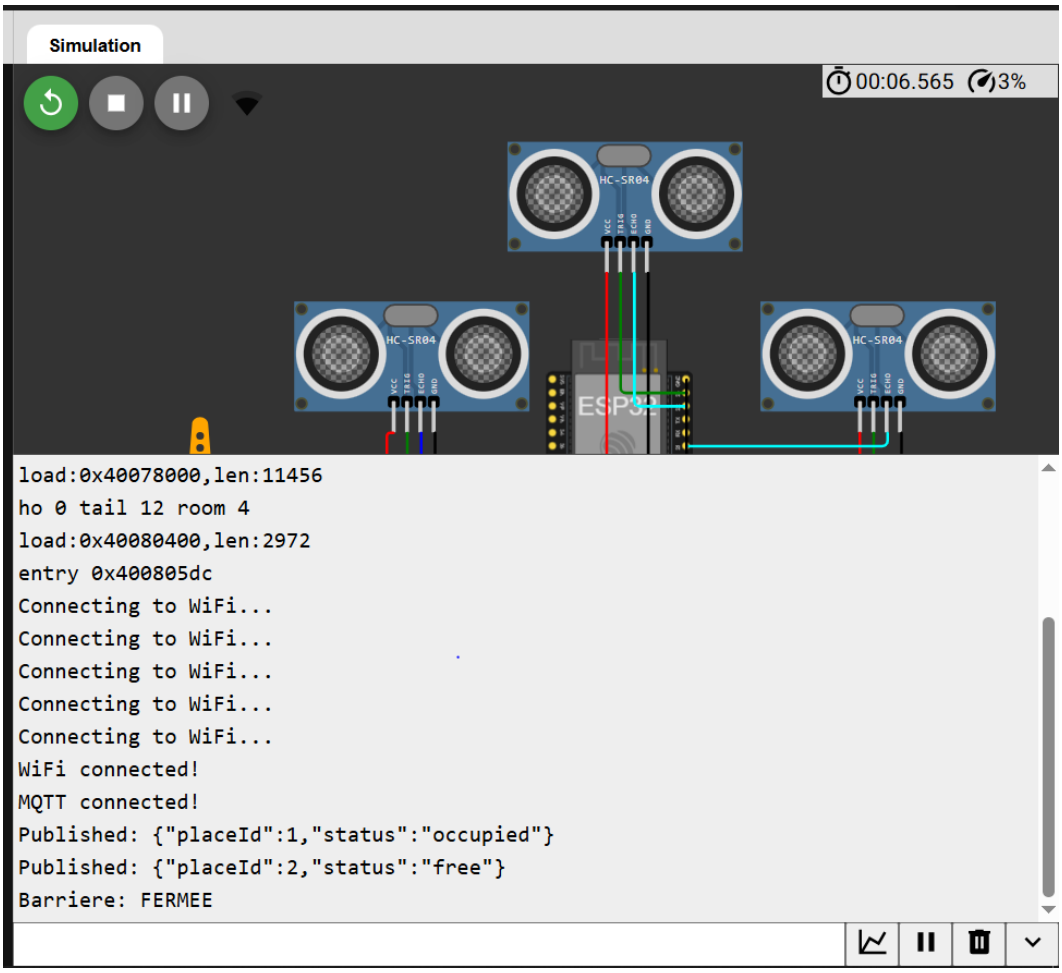

FIGURE 3.2 – Simulation en cours – Serial Monitor avec messages MQTT publies

Chapitre 4

Developpement du Firmware ESP32

4.1 Structure du Code

1. **Bloc 1** : Librairies et configuration
2. **Bloc 2** : `setup()` – Connexion WiFi, MQTT, PWM
3. **Bloc 3** : Fonctions utilitaires
4. **Bloc 4** : `loop()` – Logique principale

4.2 Librairies et Configuration

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid        = "Wokwi-GUEST";
const char* mqtt_server = "broker.hivemq.com";
const int   mqtt_port   = 1883;
const char* mqtt_topic  = "parking/update";

#define TRIG1 5
#define ECHO1 18
#define TRIG2 19
#define ECHO2 21
#define TRIG3 23
#define ECHO3 22
#define SERVO_PIN 27

const int THRESHOLD = 10;
```

Listing 4.1 – Declaration des librairies et broches

4.3 Fonction de Mesure de Distance

```
long readDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000);
  return duration * 0.034 / 2;
}
```

Listing 4.2 – Fonction readDistance()

4.4 Logique Principale

```
void loop() {
  if (!client.connected()) connectMQTT();
  client.loop();

  long d1    = readDistance(TRIG1, ECHO1);
  long d2    = readDistance(TRIG2, ECHO2);
  long dGate = readDistance(TRIG3, ECHO3);

  String status1 = (d1 < THRESHOLD) ? "occupied" : "free";
  String status2 = (d2 < THRESHOLD) ? "occupied" : "free";

  String msg1 = "{\"placeId\":1,\"status\":\"" + status1 +
     "\"}";
  String msg2 = "{\"placeId\":2,\"status\":\"" + status2 +
     "\"}";
  client.publish(mqtt_topic, msg1.c_str());
  client.publish(mqtt_topic, msg2.c_str());

  bool carAtGate = (dGate < THRESHOLD);
  if ((d1 >= THRESHOLD || d2 >= THRESHOLD) && carAtGate) {
    setServoAngle(90);
  } else {
    setServoAngle(0);
```

```
  }
  delay(3000);
}
```

Listing 4.3 – Boucle principale loop()

4.5 Test de la Barriere

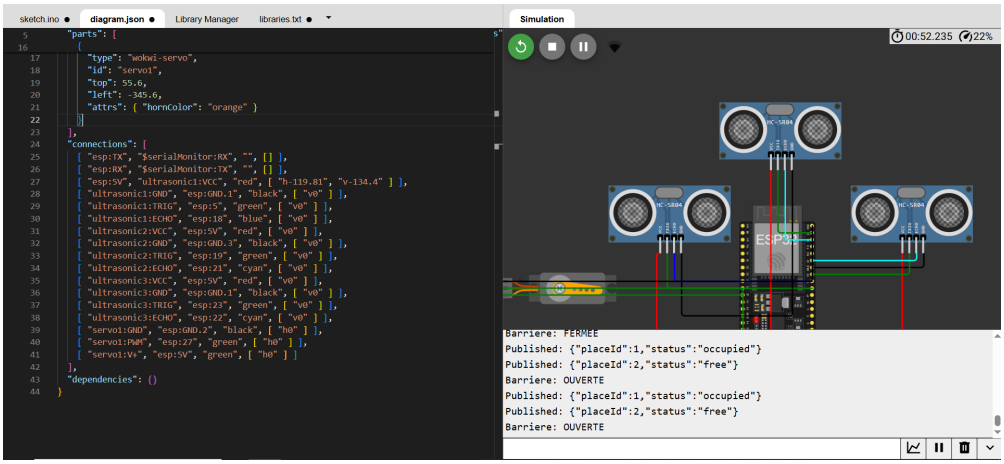

FIGURE 4.1 – Test : place libre + voiture a la barriere – Servo 90 degres (OUVERT)

Chapitre 5

Communication MQTT

5.1 Presentation du Protocole MQTT

MQTT (Message Queuing Telemetry Transport) est un protocole de messagerie leger, base sur le modele **publish**/**subscribe**, ideal pour les applications IoT. Ses avantages :

— Protocole leger (overhead minimal)

— Fonctionne sur connexions instables

— Architecture decoupplee (publish/subscribe)

— Support de QoS (Quality of Service)

5.2 Configuration HiveMQ

Parametre	Valeur
Broker	`broker.hivemq.com`
Port TCP	`1883`
Port WebSocket	`8884`
Topic	`parking/update`

TABLE 5.1 – Configuration MQTT HiveMQ

5.3 Format des Messages

Listing 5.1 – Format des messages MQTT

```
{"placeId": 1, "status": "occupied"}
{"placeId": 2, "status": "free"}
```

5.4 Visualisation des Messages

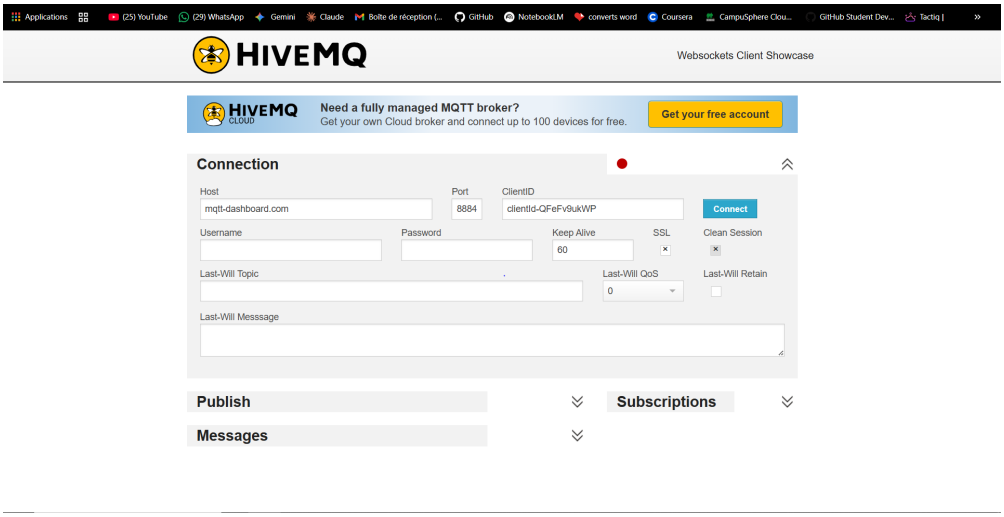

FIGURE 5.1 – Interface HiveMQ WebSocket Client

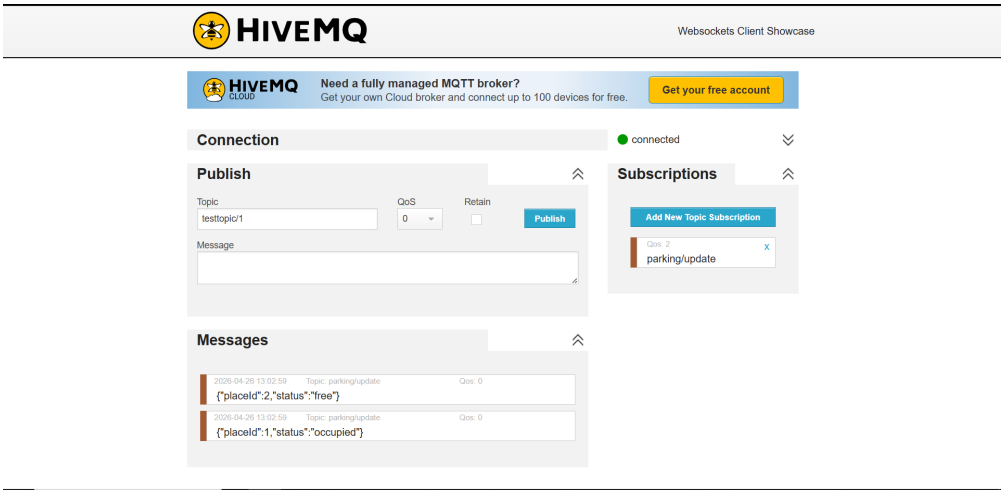

FIGURE 5.2 – Messages MQTT recus en temps reel sur le topic `parking/update`

Chapitre 6

Backend – Docker, MongoDB et Redis

6.1 Architecture Docker

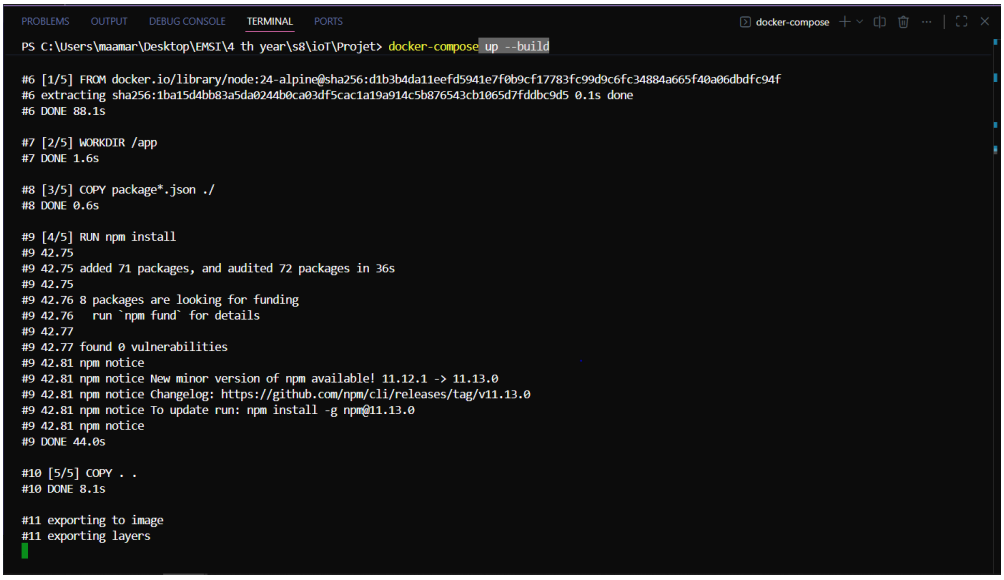

FIGURE 6.1 – Fichier docker-compose.yml – Orchestration des services

Listing 6.1 – Lancement de l'infrastructure Docker

```
docker-compose up --build
```

6.2 Services Docker

Service	Image	Port	Role
parking_mongodb	mongo :latest	27018 :27017	Stockage des donnees
parking_redis	redis :latest	6379 :6379	Cache temps reel
parking_backend	Node.js custom	3000 :3000	Traitement MQTT

TABLE 6.1 – Services Docker Compose

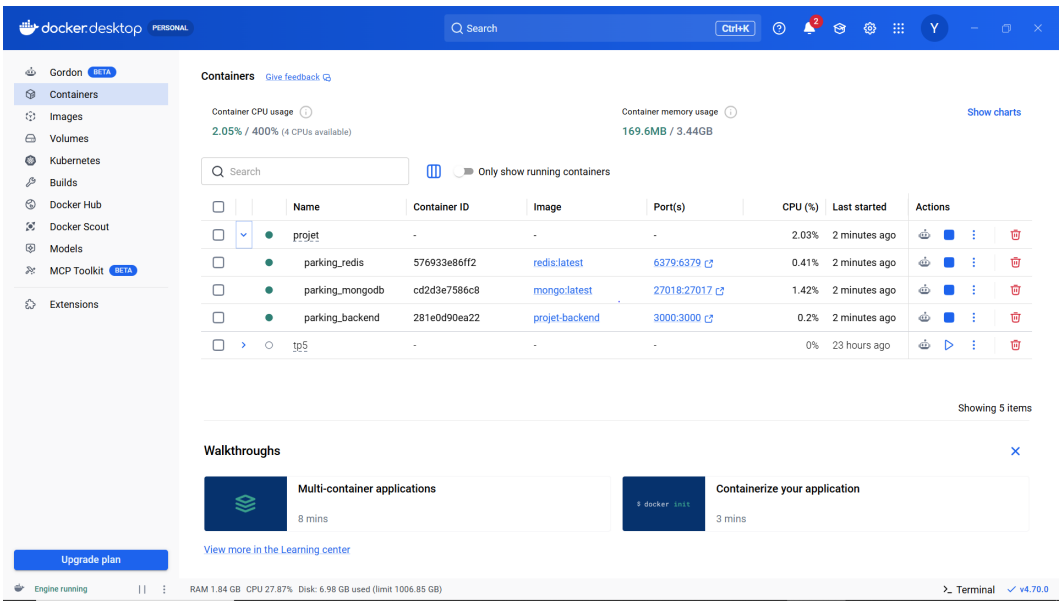

FIGURE 6.2 – Docker Desktop – Les 3 containers en cours d'execution

6.3 Resultats

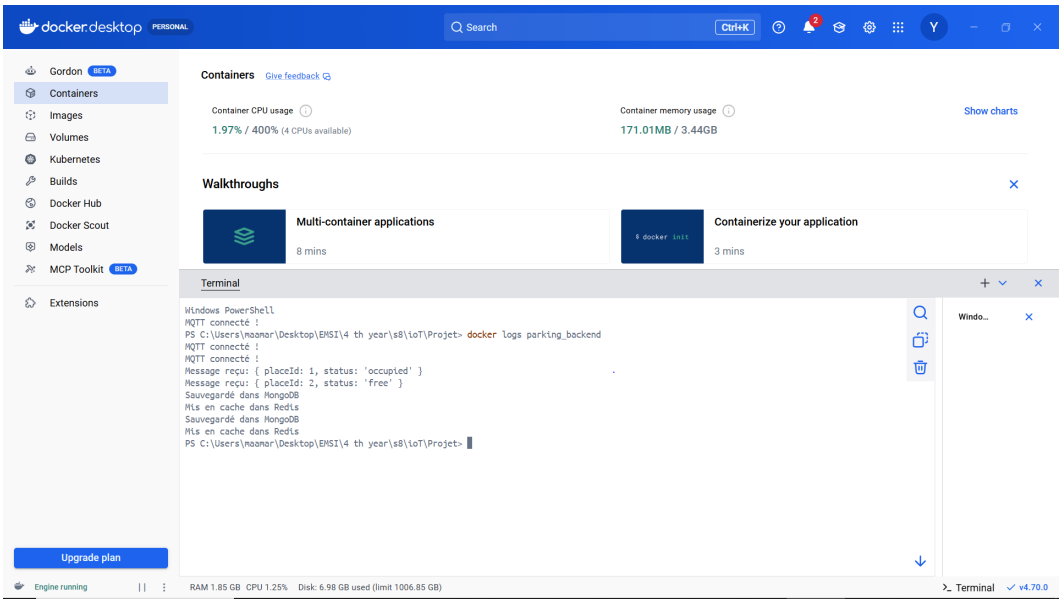

FIGURE 6.3 – Logs Docker – Messages MQTT recus, sauvegardes dans MongoDB et Redis

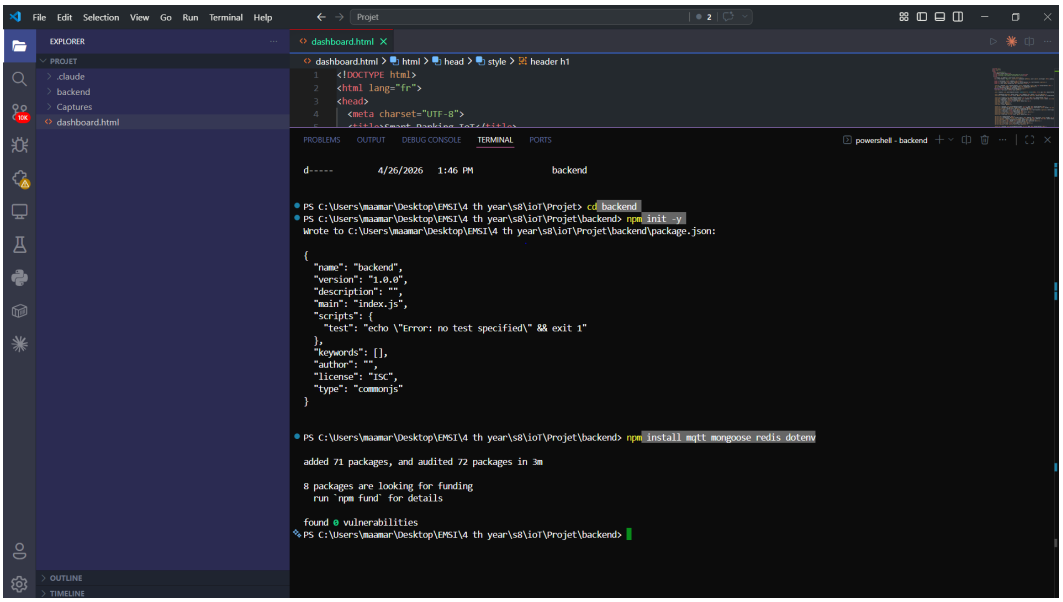

FIGURE 6.4 – Donnees sauvegardees dans MongoDB

Chapitre 7

Dashboard Web de Visualisation

7.1 Presentation

Le dashboard est developpe en **HTML/CSS/JavaScript** avec **Chart.js**. Il affiche en temps reel :

— L'etat de chaque place (LIBRE / OCCUPEE)

— L'etat de la barriere (OUVERTE / FERMEE)

— Un graphique lineaire de l'historique d'occupation

— Un graphique donut de la repartition actuelle

— Un log des derniers messages MQTT

7.2 Vue Generale

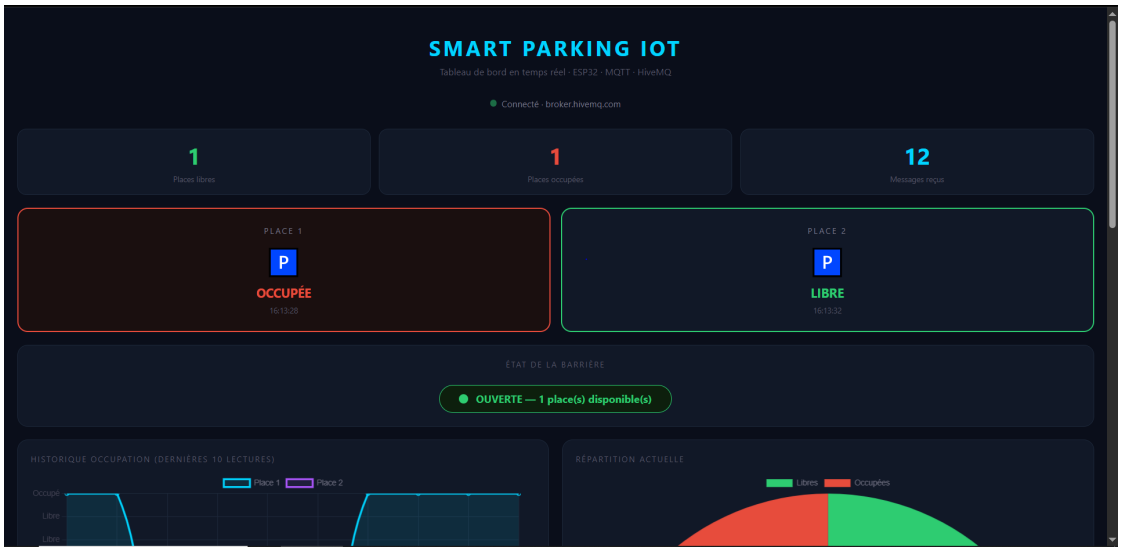

FIGURE 7.1 – Dashboard Smart Parking IoT – Vue principale

7.3 Graphiques

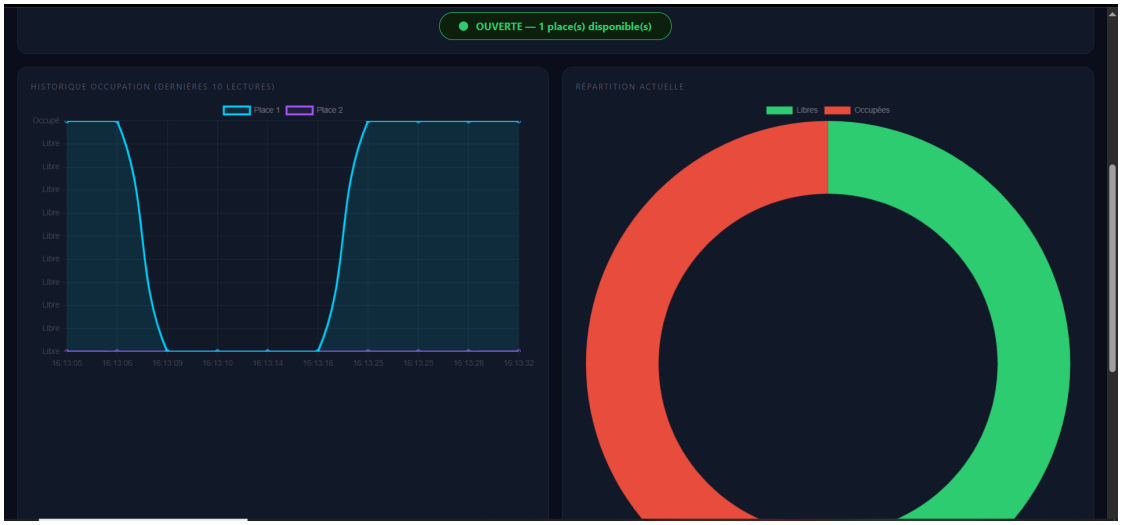

FIGURE 7.2 – Dashboard – Historique d'occupation et repartition actuelle

7.4 Fonctionnalites

Fonctionnalite	Description
Connexion temps reel	Via WebSocket MQTT (HiveMQ port 8884)
Etat des places	Cartes rouge/verte selon l'occupation
Etat barriere	Indicateur OUVERTE/FERMEE dynamique
Historique	Graphe lineaire des 10 dernieres lectures
Repartition	Graphe donut libre/occupe
Log MQTT	6 derniers messages avec horodatage

TABLE 7.1 – Fonctionnalites du Dashboard

Chapitre 8

Integration RAG-LLM

8.1 Presentation du RAG

Le systeme integre un module d'intelligence artificielle base sur l'architecture **RAG** (Retrieval-Augmented Generation), permettant des recommandations intelligentes a partir des donnees historiques du parking.

8.2 Architecture RAG

```
MongoDB (historique parking)
        |
   getContext()
   - Lit les 20 dernieres entrees
   - Calcule le taux d'occupation
   - Formate en texte structure
        |
   OpenRouter API
   Modele : meta-llama/llama-3.2-3b-instruct:free
        |
   Recommandations en francais
```

8.3 Implementation

```
1 async function getContext() {
2   const data = await Parking.find()
3     .sort({ timestamp: -1 })
4     .limit(20);
5
```

```
  const occupied = data.filter(d => d.status === 'occupied').
     length;
  const free     = data.filter(d => d.status === 'free').length
     ;
  const total    = data.length;

  return `
    Total de lectures : ${total}
    Places occupees   : ${occupied}
    Places libres     : ${free}
    Taux occupation   : ${Math.round((occupied/total)*100)}%
  `;
}
```

Listing 8.1 – Fonction getContext() – Extraction du contexte depuis MongoDB

8.4 Technologies IA

Composant	Detail
API	OpenRouter.ai
Modele	meta-llama/llama-3.2-3b-instruct :free
Librairie	axios (Node.js)
Securite	Cle API dans fichier `.env`

TABLE 8.1 – Configuration du module RAG-LLM

Securite

La cle API OpenRouter est stockee dans un fichier `.env` exclu du depot GitHub via `.gitignore`, garantissant la confidentialite des credentials.

Chapitre 9

Conclusion et Perspectives

9.1 Bilan du Projet

- √ **Couche physique** : 3 capteurs HC-SR04 + servo moteur + ESP32
- √ **Couche communication** : WiFi + MQTT + HiveMQ
- √ **Couche stockage** : MongoDB (persistant) + Redis (cache)
- √ **Couche deploiement** : Docker Compose (3 containers)
- √ **Couche visualisation** : Dashboard HTML temps reel avec Chart.js
- √ **Couche intelligence** : RAG-LLM via OpenRouter

9.2 Competences Acquises

— Programmation embarquee (ESP32, Arduino C++)

— Protocoles IoT (MQTT, WiFi)

— Developpement backend (Node.js)

— Bases de donnees NoSQL (MongoDB, Redis)

— Conteneurisation (Docker, Docker Compose)

— Developpement frontend (HTML, CSS, JavaScript, Chart.js)

— Intelligence Artificielle (RAG, LLM, OpenRouter)

9.3 Perspectives et Ameliorations

Priorite	Amelioration
Haute	API REST pour lire l'historique MongoDB depuis le dashboard
Haute	LEDs rouge/vert pour indiquer l'etat de chaque place
Moyenne	Ecran LCD affichant le nombre de places disponibles
Moyenne	Notifications Telegram en cas de parking complet
Basse	Authentification securisee sur le dashboard
Future	Analyse predictive avec ML pour anticiper l'occupation

TABLE 9.1 – Perspectives d'amelioration

9.4 Conclusion

Ce projet demontre la puissance de l'IoT moderne pour resoudre des problemes concrets. En combinant des technologies open-source — ESP32, MQTT, MongoDB, Redis, Docker et l'IA — il est possible de construire des systemes intelligents robustes et scalables. Le systeme developpe constitue une base solide pour un deploiement en conditions reelles.

Smart Parking IoT System

EMSI Casablanca — S8 — 2025/2026

Bibliographie

[1] Espressif Systems, *ESP32 Technical Reference Manual*, Version 5.1, 2023. https://www.espressif.com/

[2] OASIS Standard, *MQTT Version 5.0 Specification*, 2019. https://mqtt.org/

[3] HiveMQ GmbH, *HiveMQ MQTT Broker Documentation*, 2024. https://www.hivemq.com/

[4] MongoDB Inc., *MongoDB Manual*, 2024. https://www.mongodb.com/docs/

[5] Redis Labs, *Redis Documentation*, 2024. https://redis.io/docs/

[6] Docker Inc., *Docker Documentation*, 2024. https://docs.docker.com/

[7] Wokwi, *Wokwi ESP32 Simulator*, 2024. https://wokwi.com/

[8] OpenRouter, *OpenRouter API Documentation*, 2024. https://openrouter.ai/docs